SALACYBER

Web Security Foundation

MODULE 1

Web Fundamentals and HTTP

4 Sessions · 8 Hours

HTTP | DNS | DevTools | Wireshark | curl | Postman

© SalaCyber Web Security Foundation · SWSF

What We Cover in Module 1

4 sessions · 8 hours · 14 lessons + 4 labs

1

How the Web Works

Client-Server · DNS · URLs

2

HTTP Protocol Deep Dive

Requests · Responses · Status Codes

3

HTTP Headers & Methods

Headers · Methods · DevTools

4

Review & Mini-Project

Recap · Hands-on · Q&A

By the end of this module, you'll understand how web communication works at the HTTP level — the foundation of all web security.

How Web Development Evolved

From static pages to APIs — understanding the history explains why HTTP works the way it does

Key insight: The shift from full-page reloads → partial updates → API-driven apps is WHY modern HTTP and security patterns look the way they do. Every generation added new attack surfaces.

1991–1999

Web 1.0

Static HTML

- Pure HTML files on a server
- No interactivity — read-only
- Full page reload for every action
- URL = a file on disk

2000–2004

Server-Side Forms

CGI & PHP Era

- HTML <form> submits to server
- Server processes & returns NEW page
- Entire page reloads on every submit
- PHP, ASP, JSP handle logic

2005–2012

AJAX Revolution

jQuery & AJAX

- JS sends requests in background
- Only part of page updates
- XMLHttpRequest (XHR) born
- Web apps feel like desktop apps

2013–Now

Modern SPA + API

React / Vue + REST API

- Frontend = JS app (React, Vue)
- Backend = pure JSON API
- fetch() / axios replaces XHR
- Complete separation of concerns

The Web Technology Stack

How HTML, CSS, JavaScript, and the server all fit together

SALACYBER

BROWSER (Client Side)

HTML

Structure & Semantics

`<h1>` `<p>` `<form>` `<table>`

Defines WHAT is on the page — headings, paragraphs, links, inputs. The skeleton.

CSS

Presentation & Layout

`color: blue;` `display: flex;`

Defines HOW it looks — fonts, colors, spacing, animations. The skin.

JavaScript

Behavior & Interaction

`fetch()` `addEventListener()` `DOM`

Defines WHAT IT DOES — click handlers, form validation, API calls. The muscles.

→
←

HTTP/HTTPS

SERVER (Back-End)

Web Server

Nginx / Apache

Routes requests, serves static files, TLS termination

App Framework

Flask / Django / Express

Business logic, routing, auth, APIs

Database

PostgreSQL / MySQL / SQLite

Persistent data storage and querying

SESSION 1

How the Web Works

Internet architecture · DNS · URL structure
Client-server model

SALACYBER

What Happens When You Type a URL?

Session 1 · Internet & Web Architecture

SALACYBER

1	2	3	4	5	6
You type salacyber.com	Browser asks DNS server	DNS returns IP address	TCP connection to server	HTTP Request sent	Server sends HTTP Response

Key Concepts

Client — Your browser (Chrome, Firefox). Initiates all requests.

Server — Listens on a port (80/443), processes requests, returns responses.

Protocol — Agreed-upon rules: HTTP defines message format and behavior.

"The web is just computers asking each other questions and getting answers."

— A useful mental model for every topic in this course

DNS: The Internet's Phone Book

Session 1 · Domain Name System Basics

Browser Cache Check → Recursive Resolver → Root Nameserver → TLD Server (.com, .org) → Auth. Nameserver

Returns IP address: 123.123.80.46

Common DNS Record Types

A	AAAA	CNAME	MX	TXT
Maps domain → IPv4 address	Maps domain → IPv6 address	Alias for another domain	Mail server for domain	Text data (SPF, DKIM, etc.)

The /etc/hosts File

Session 1 · Your computer's local DNS override — checked BEFORE any DNS query is made

What it is

A plain text file that maps hostnames to IP addresses locally, before DNS is consulted.

Lookup order

OS checks hosts file FIRST → then DNS cache → then DNS resolver. A match in hosts file wins every time.

Location

Linux / macOS: /etc/hosts
Windows: C:\Windows\System32\drivers\etc\hosts

Common uses

Block ads/trackers · Local dev (myapp.local) · Override a domain for testing · Penetration testing

```
# /etc/hosts  (Linux / macOS)

127.0.0.1       localhost
::1             localhost

# Block tracking domain
0.0.0.0         ads.doubleclick.net

# Local dev environment
127.0.0.1       myapp.local
127.0.0.1       api.myapp.local
```

Security Relevance

- Malware modifies hosts file to redirect banking sites to fake IPs
- Attackers can hijack example.com → their phishing server
- Penetration testers use it to test apps before DNS is configured
- Always check hosts file when debugging 'wrong site is loading'

URL Anatomy

Session 1 · URL and Web Resource Structure

SALACYBER

```
https://api.shop.com:443/products/shoes?color=red&size=42#reviews
```

Scheme

`https://`

Port

`:443`

Query String

`?color=red&size=42`

Host

`api.shop.com`

Path

`/products/shoes`

Fragment

`#reviews`

Security tip: Always validate and sanitize URL parameters server-side — query strings are user-controlled input!

Ports & Common Protocols

Session 1 · Every network service lives on a port number

SALACYBER

A port is like an apartment number. The IP address is the building — the port tells the network WHICH service to talk to.

Port	Protocol	Encrypted?	What it's used for
80	HTTP	⚠ No	Plain web traffic. Anything on this port is unencrypted.
443	HTTPS	🔒 Yes	Encrypted web traffic. TLS wraps HTTP. Default for all modern sites.
22	SSH	🔒 Yes	Secure shell — encrypted remote terminal access to servers.
21	FTP	⚠ No	File Transfer Protocol — unencrypted. Never use over public networks.
25	SMTP	⚠ No	Email sending. Often blocked by ISPs to prevent spam bots.
3306	MySQL	⚠ No	Database port. NEVER expose to internet — huge attack surface.
5432	PostgreSQL	⚠ No	PostgreSQL DB port. Same rule — firewall it, never expose publicly.
8080	HTTP Alt	⚠ No	Developers run apps here to avoid requiring root for port 80.

Ports 0–1023 are 'well-known' and require root/admin to open. Ports 1024–65535 are available to regular apps.

SALACYBER

Inspect HTTP Requests Using Browser DevTools

1. Open Chrome/Firefox DevTools → Network tab (F12 → Network)
2. Visit 3 different sites: a news site, a bank site, and a social media site
3. For each site, capture at least 5 requests. For each one: identify the method, URL, status code, and key headers
4. Filter by 'XHR/Fetch' — these are the API calls. How many can you spot?
5. Find a request that sets a cookie. What flags does it have? (HttpOnly? Secure?)
6. Bonus: find a redirect chain (3xx → 200) and trace the full path

Keep DevTools open throughout the course — it's your most powerful learning tool.

SESSION 2

HTTP Protocol Deep Dive

HTTP overview · Request structure
Response structure · Status codes

SALACYBER

What is HTTP?

Session 2 · HTTP Protocol Overview

Text-Based

HTTP messages are human-readable ASCII text — you can read them in Wireshark.

Stateless

Each request is independent. The server remembers nothing about previous requests.

Request-Response

Every interaction is a client request followed by exactly one server response.

Application Layer

HTTP sits on top of TCP/IP. It doesn't care how the bytes get there — just what they say.

Stateless vs Stateful

Session 2 · One of the most important concepts in web security

SALACYBER

STATELESS

HTTP is stateless by design

👨‍🍳 Analogy: A waiter with amnesia

Every time you call the waiter, he has forgotten who you are, what you ordered before, and whether you already paid. Each request is brand new.

→ Each HTTP request carries ALL info needed (headers, tokens, cookies)

→ Server holds zero memory between requests

→ Scales easily — any server can handle any request

→ Simplifies server design but shifts burden to client

STATEFUL

Achieved on top of HTTP via sessions/tokens

👨‍🍳 Analogy: A waiter with a notepad

The waiter writes your name and order down. Next visit he recognises you and remembers your last order. State is maintained between interactions.

→ Server stores user session data (in DB or memory)

→ Client gets a session ID cookie — presented each request

→ More complex, harder to scale horizontally

→ Examples: login sessions, shopping carts, websockets

Security implication: because HTTP is stateless, every mechanism for 'remembering' users (cookies, tokens, sessions) is an attack surface you must protect.

HTTP vs HTTPS

Session 2 · HTTP Protocol Overview

SALACYBER

HTTP (Port 80)

- Plain text — everything visible
- No encryption in transit
- Susceptible to MITM attacks
- No certificate verification
- Fast (no TLS handshake)

Sending login credentials over HTTP? Captured instantly with Wireshark.

HTTPS (Port 443)

- Encrypted via TLS (1.2 / 1.3)
- Certificate proves server identity
- Protects against MITM
- Required for cookies w/ Secure flag
- Required for HTTP/2 in practice

Rule: Never send sensitive data over HTTP. Always enforce HTTPS in production.

HTTP Versions — How the Protocol Evolved

Session 2 · HTTP/1.0 → HTTP/1.1 → HTTP/2 → HTTP/3

SALACYBER

HTTP/1.0

1996 · Legacy

- One TCP connection per request
- Connection closed after each response
- Extremely slow for pages with many resources
- No persistent connections, no pipelining

```
GET /index.html HTTP/1.0
Connection: close
```

Essentially extinct. You may see it in old logs.

HTTP/1.1

1997 · Still common

- Persistent connections (keep-alive) by default
- Pipelining (send multiple requests, wait for responses)
- Host header becomes mandatory
- Chunked transfer encoding for streaming

```
GET /index.html HTTP/1.1
Host: example.com
Connection: keep-alive
```

Still used everywhere. Default for most curl/Postman requests.

HTTP/2

2015 · Modern

- Binary framing (not human-readable text)
- Multiplexing: many requests over ONE connection
- Header compression (HPACK) — smaller overhead
- Server Push — server can pre-send resources

```
# Binary protocol — not human
readable
# Requires HTTPS in practice
# Multiplexed streams over 1 TCP
conn
```

Used by ~65% of websites. Requires HTTPS. Major performance win.

HTTP/3

2022 · Cutting edge

- Runs over QUIC (UDP-based, not TCP)
- Eliminates TCP head-of-line blocking
- Built-in encryption (TLS 1.3 in QUIC)
- Faster connection establishment (0-RTT)

```
# Built on QUIC/UDP
# TLS built in, not layered on top
# Alt-Svc: h3=":443"
```

~30% of web traffic. Used by Google, Facebook, Cloudflare.

Anatomy of an HTTP Request

Session 2 · HTTP Request Structure

```
POST /api/login HTTP/1.1

Host: api.example.com

Content-Type: application/json

Authorization: Bearer eyJhbGci...

User-Agent: Mozilla/5.0 (Windows NT 10.0)

Accept: application/json

Cookie: session=abc123; theme=dark



{ "username": "alice", "password": "..." }
```

Request Line —
Method + Path + HTTP version

Host Header —
Required in HTTP/1.1

Content-Type —
Format of the request body

Authorization —
Auth credential (bearer token)

User-Agent —
Browser/client identification

Cookie —
Session data sent back to server

Body —
JSON payload (POST/PUT only)

Anatomy of an HTTP Response

Session 2 · HTTP Response Structure

```
HTTP/1.1 200 OK

Content-Type: application/json; charset=utf-8

Content-Length: 142

Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Strict

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000

Cache-Control: no-store



{ "token": "eyJhbGci...", "user": "alice" }
```

Status Line

HTTP version + status code + reason

Content-Type

Format of the body (MIME type)

Set-Cookie

HttpOnly & Secure flags protect sessions

Security Headers

nosniff, HSTS — more in Module 6

Body

The actual response data (HTML, JSON, etc.)

HTTP Status Codes

Session 2 · Response Status Codes Reference

SALACYBER

2xx
Success

- 200 OK
- 201 Created
- 204 No Content

3xx
Redirect

- 301 Moved Permanently
- 302 Found
- 304 Not Modified

4xx
Client Error

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 429 Too Many Requests

5xx
Server Error

- 500 Internal Server Error
- 502 Bad Gateway
- 503 Service Unavailable

SALACYBER

Postman, curl & Building an HTML Form

1. Lab 2a — Postman: Send GET to https://jsonplaceholder.typicode.com/posts/1. Examine response headers and body
2. Lab 2b — curl: Run: curl -v https://jsonplaceholder.typicode.com/posts -X POST -H 'Content-Type: application/json' -d '{"title":"test"}'
3. Lab 2c — Compare: what does -v show that Postman's UI hides? Why does that matter for security?
4. Lab 3 — Build an HTML form with two inputs (username, password). Add method=GET first, submit and observe the URL. Then switch to method=POST — what changes? Why is POST safer for sensitive data?

In curl, -v (verbose) shows the full request and response including headers — just like Wireshark but for a single request.

SESSION 3

HTTP Headers & Methods

Request & response headers · HTTP methods
DevTools traffic inspection

SALACYBER

Key HTTP Request Headers

Session 3 · HTTP Request Headers

SALACYBER

Host: `api.example.com`

Required. Identifies which virtual host to route to. Critical for shared hosting.

User-Agent: `Mozilla/5.0 (Windows NT 10.0; Win64)`

Identifies the browser/client. Can be spoofed — never trust for security decisions.

Accept: `application/json, text/html;q=0.9`

Tells server what content types the client understands (MIME negotiation).

Authorization: `Bearer eyJhbGciOiJIUzI1Ni...`

Carries auth credentials. Never put in URL query string — URLs are logged!

Cookie: `session=abc123; csrf_token=x9f2`

Auto-sent by browser. Session cookies here = why HTTPS is critical.

Content-Type: `application/json`

Tells server the format of the request body. Mismatch causes 415 errors.

Common HTTP Response Headers

Session 3 · What the Server Sends Back

SALACYBER

Content-Type: `application/json; charset=utf-8`

Tells the browser how to interpret the body. If missing/wrong, browsers may sniff type — a security risk.

Set-Cookie: `session=abc; HttpOnly; Secure; SameSite=Strict`

Sets a cookie. HttpOnly = JS can't read it (blocks XSS theft). Secure = HTTPS only. SameSite = blocks CSRF.

Location: `https://www.example.com/new-path`

Used with 3xx redirects. Browser follows this URL automatically.

Cache-Control: `no-store, max-age=3600`

Controls browser/proxy caching. no-store for sensitive pages (banking, auth). max-age for static assets.

Strict-Transport-Security: `max-age=31536000; includeSubDomains`

HSTS: forces HTTPS for 1 year. Prevents protocol downgrade attacks. Only works over HTTPS.

X-Content-Type-Options: `nosniff`

Stops browser from guessing content type. Prevents MIME-type confusion attacks.

HTTP Methods — Full Reference

Session 3 · When to use each method, and why it matters for security

SALACYBER

Method	Body?	Safe?	Idempotent?	Cacheable?	Common Use-case + Security Note
GET	No	✓	✓	✓	Retrieve data. Never change state via GET — it can be cached, bookmarked, and pre-fetched by browsers.
POST	Yes	✗	✗	✗	Create a resource. Double-submit danger — add idempotency keys for payments. Body carries the data.
PUT	Yes	✗	✓	✗	Full replace. Safe to retry on network failure (idempotent). Requires sending the complete resource.
PATCH	Yes	✗	✗	✗	Partial update. Only send changed fields. Not always idempotent — test your API carefully.
DELETE	No	✗	✓	✗	Remove resource. Idempotent — deleting twice returns 404, not an error. Require authorization!
OPTIONS	No	✓	✓	✗	Browser CORS preflight. Server declares allowed methods/origins. Happens automatically before cross-origin fetch.
HEAD	No	✓	✓	✓	Same as GET but returns headers only — no body. Used to check if resource exists or get Content-Length.

Safe = does not modify server state · Idempotent = calling N times has same effect as calling once · ✓ = Yes · ✗ = No

HTTP Methods

Session 3 · GET · POST · PUT · PATCH · DELETE · OPTIONS

GET

Retrieve data. No body. Should NEVER change server state.

POST

Create a resource. Body required. NOT idempotent — avoid double submit.

PUT

Replace a resource entirely. Idempotent — calling twice = same result.

PATCH

Partial update. Only send the fields you want to change.

DELETE

Remove a resource. Should be idempotent — deleting twice = no error.

HEAD & OPTIONS

HEAD: Similar to GET but only ask for HTTP headers not the response body.

OPTIONS: Preflight for CORS. Browser sends this before cross-origin requests.

Cookies & Sessions — Simulating State over HTTP

Session 3 · How websites remember who you are

SALACYBER

What is a Cookie?

```
Set-Cookie: session=abc123; HttpOnly; Secure;
        SameSite=Strict; Max-Age=3600; Path=/
```

HttpOnly	JS cannot access this cookie via document.cookie — blocks XSS session theft
Secure	Only sent over HTTPS — prevents interception on plain HTTP connections
SameSite	Controls cross-site sending: Strict/Lax/None — primary CSRF defense
Max-Age	Cookie lifetime in seconds. Omit for session cookie (deleted on browser close)
Path=/	Which URL paths receive the cookie. / means all paths on the domain

Session Flow: Login → Logout

1. **User POSTs credentials**
   `POST /login { username, password }`
2. **Server verifies, creates session**
   `Stores session_id → user data in DB`
3. **Server sets cookie**
   `Set-Cookie: sid=abc; HttpOnly; Secure`
4. **Browser stores cookie**
   `Auto-sent with EVERY subsequent request`
5. **Server validates session ID**
   `Looks up session in DB → identifies user`
6. **Logout: server destroys session**
   `DELETE session from DB + clear cookie`

CORS — Cross-Origin Resource Sharing

Session 3 · Why browsers block some API calls — and how CORS unlocks them

Same-Origin Policy (SOP): Browsers block JS on page A from reading responses from page B if they have different origins (scheme + domain + port). CORS is the mechanism to selectively relax this rule.

From (browser page)	Requesting	Same Origin?	Reason
`https://app.example.com/page`	`https://app.example.com/api/data`	✓ Yes	Same scheme, same domain, same port
`https://app.example.com/page`	`https://api.example.com/data`	✗ No	Different subdomain
`https://app.example.com/page`	`http://app.example.com/data`	✗ No	Different scheme (https vs http)
`https://app.example.com/page`	`https://app.example.com:8080/data`	✗ No	Different port

CORS Response Headers — server grants permission:

Access-Control-Allow-Origin:	`https://app.example.com`	← *Which origin is allowed (or * for all — avoid on auth endpoints)*
Access-Control-Allow-Methods:	`GET, POST, PUT, DELETE`	← *Which HTTP methods are allowed*
Access-Control-Allow-Headers:	`Authorization, Content-Type`	← *Which request headers are allowed*
Access-Control-Allow-Credentials:	`true`	← *Allow cookies/auth headers — NEVER combine with Allow-Origin: **

Inspecting Web Traffic with DevTools

Session 3 · Network Tab Deep Dive

SALACYBER

Elements | Console | Sources | **Network** | Performance | Application

Preserve log Disable cache | Filter: XHR JS CSS Img Media Font Doc WS Wasm Other

Name	Status	Type	Initiator	Size	Time
api/login	200	xhr	app.js:42	1.2 kB	128 ms
user/profile	200	fetch	auth.js:15	3.4 kB	95 ms
api/posts	404	xhr	feed.js:88	0.4 kB	62 ms
analytics.js	200	script	index.html	42.1 kB	230 ms
track	302	xhr	analytics.js:1	0 B	18 ms

Click any row → Headers tab: see full request/response headers. Response tab: see the body. Timing tab: see where time was spent.

SALACYBER

Capture and Analyze HTTP Traffic with Wireshark / tcpdump

1. Start Wireshark → select your active network interface → Apply filter: http
2. Open a browser and visit http://neverssl.com (a plain HTTP site — perfect for analysis)
3. In Wireshark: right-click a request → Follow → HTTP Stream. Read the entire conversation in plain text
4. Try to find: the request Host header, the response Content-Type, and any Set-Cookie headers
5. Now visit https://google.com with filter: tls — observe: you can see the TLS handshake but NOT the content. This is why HTTPS matters.
6. Bonus — tcpdump: sudo tcpdump -i eth0 -A port 80 and observe captured traffic directly in terminal

Wireshark on HTTP is a powerful demonstration: students see exactly what attackers on a network can capture.

SESSION 4

Review & Mini-Project

Cheat sheet recap · Peer review
Mini-project: craft an HTTP request

SALACYBER

Module 1 Cheat Sheet

Session 4 · Quick Reference

SALACYBER

Status Codes

200	OK
201	Created
204	No Content
301	Moved Permanently
302	Found
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Server Error

HTTP Methods

GET	Read, safe, idempotent
POST	Create, not idempotent
PUT	Replace, idempotent
PATCH	Partial update
DELETE	Remove, idempotent
OPTIONS	CORS preflight

Security Reminders

- Always use HTTPS in production
- Never put secrets in URLs
- Passwords → POST body, never GET
- Check HttpOnly & Secure on cookies
- Don't trust User-Agent for security
- Validate query params server-side
- Log requests (not passwords!)

Mini-Project: Craft a Raw HTTP Request

Session 4 · 45-min hands-on + class demo

SALACYBER

01

Write the Request

In a text file, manually write a valid HTTP/1.1 POST request. Include:

- Request line (method, path, version)
- Host, Content-Type, Authorization headers
- A JSON body with at least 2 fields

02

Send It with curl

Use curl to send your request to

https://httpbin.org/post

Capture the full response with -v flag

Verify the server echoes your headers & body

03

Analyze & Present

Present to the class (2 min each):

- Show your request in the text file
- Show the curl output in terminal
- Explain one header you included and WHY

Module 1 Complete!

You can now:

- Explain the client-server model and DNS resolution process
- Break down a URL into its components
- Read and write HTTP request and response messages
- Identify HTTP methods and when to use each
- Interpret status codes and response headers
- Capture and analyze HTTP traffic with DevTools & Wireshark

SALACYBER

NEXT: Module 2 — APIs and Web Communication